ZIMBABWEAN ASTRONOMICAL SOCIETY · DATA SCIENCE REGIONAL HACKATHON 2026

Dating Cities from Space

Building-age classification from 42 years of Landsat imagery

Madrid → Amsterdam

Team Guide — everything we did, explained simply

For every team member, with or without a data-science background:
what the challenge is, how we solved it step by step, with code,
why we kept our results, and how to present and defend them.

Group 2

Zimbabwe National Geospatial and Space Agency (ZINGSA) · 5–8 October 2026

Organised by the Zimbabwean Astronomical Society with AfAS, OAD, IDIA and DARA; hosted by ZINGSA.

Contents

1. Read this first — the 60-second story and the numbers to remember
2. The challenge in plain words
3. Key words, explained simply
4. Stage 1 — Understanding the data
5. Stage 2 — Cleaning the data
6. Stage 3 — Feature engineering (what we added and why)
7. Stage 4 — Training the Madrid model
8. Stage 5 — Moving the model to Amsterdam
9. Stage 6 — Testing ideas from research papers
10. Final results, and why we keep them
11. Questions the judges may ask — with answers
12. Who presents what, slide by slide
13. Our files and how to run them
14. References

1. Read this first

This guide is for **every member of Group 2**. You do not need any data-science background. Read it once from start to finish (about 60 minutes). Each chapter follows the same pattern: **what we did**, **why**, **a small piece of the real code**, and **what came out**. Coloured boxes help you: blue explains in plain words, amber says why it matters, green gives a sentence you can say on stage, purple lists numbers to remember.

The 60-second story

The whole project in one paragraph

"Satellites have photographed every city on Earth every year since the 1980s. We used 42 years of these photos to guess how old the buildings are in each 30-metre square of a city.
We learned this in **Madrid**, where we know the answer for every square, and then moved the model to **Amsterdam**, where we pretend to know the answer for only a handful of squares.
Three ideas made the difference: we cleaned the photos better, we described **how each square changed over time** instead of how bright it is, and we let the model learn from Amsterdam's **unlabelled** squares too.
With only 25 known squares per age group, our model scores 0.687 in Amsterdam — far better than the organisers' starter baseline (0.593)."

Numbers to remember

What	Our result	Starter baseline	Meaning
Madrid score (honest test)	**0.652** ± 0.026	0.574	How well we date buildings in the training city
Amsterdam, no labels at all	**0.624**	0.334	Madrid model used directly in Amsterdam
Amsterdam, 5 labels per class	**0.654** ± 0.021	0.502	Very few examples — already as good as Madrid
Amsterdam, 25 labels per class	**0.687** ± 0.019	0.593	The "low-data prize" setting
Amsterdam, 200 labels per class	**0.757** ± 0.002	0.615	Our best score
Average of the 5 Amsterdam scores	**0.712**	0.587	The "best average" prize
Stricter check: unseen half of Amsterdam	**0.710**	—	Labels from one half of the city, scored on the other half
Features we created	**321**	60	Numbers that describe each square
Run time (laptop GPU)	**18 min**	—	Whole pipeline, raw data to results

In plain words

A score of 1.0 would mean perfect; 0.25 would be random guessing between 4 classes. "± 0.02" means the score moves by about 0.02 depending on which few examples were picked — small error bars mean the result is stable.

2. The challenge in plain words

2.1 What the satellite sees

Landsat is a family of NASA/USGS satellites that photograph the whole Earth again and again. The camera does not see single buildings: it cuts the ground into squares of **30 m × 30 m** (about two tennis courts). Each square is one **pixel**. For every pixel and every photo it measures six "colours" (bands):

Band	What it measures	Useful for
Blue, Green, Red	visible light — what our eyes see	roof and road colour
NIR (near-infrared)	invisible light; plants reflect it strongly	gardens, trees, parks
SWIR1, SWIR2 (short-wave infrared)	invisible light sensitive to materials	concrete, brick, soil, moisture

Because the satellite returns every year, each pixel has a **history of 42 years** (1984–2025). That history is the key to everything we did.

2.2 What we must predict

Each pixel gets one of four **age classes**, based on when its buildings were built. The first boundary is different in each city because it marks a big change in how that city built:

Class	Madrid	Amsterdam	What it means
1	before 1960	before 1945	old buildings — existed before the satellite record
2	1960 – 1984	1945 – 1984	mid-century buildings — also existed before 1984
3	1984 – 2004	1984 – 2004	built while the satellite was watching (first 20 years)
4	2004 – 2024	2004 – 2024	built while the satellite was watching (last 20 years)

In plain words

A pixel can contain several buildings. The organisers give one number per pixel, the **weighted mean year**: the average build year, where bigger buildings count more. Example: buildings from 1920 (300 m²), 1970 (100 m²) and 2010 (50 m²) give (1920×300 + 1970×100 + 2010×50) ÷ 450 = **1941** → class 1.

2.3 The task, step by step

- **Step 1:** train a model on **Madrid** (76,263 pixels, all labelled).
- **Step 2:** test it fairly on Madrid (cross-validation) and report the score ± its spread.
- **Step 3:** adapt the model to **Amsterdam** (25,992 pixels) using only **n** labelled pixels per class.
- **Step 4:** do this for n = 5, 25, 50, 100 and 200 and report each score ± spread, plus a plot of score against $\log_2(n)$.

The judges then test our models on **hidden pixels** we never saw. Prizes: best average score, best peak score, best score at n = 25, and originality.

2.4 How the score works: macro F1

For each class we ask two questions: **precision** — when the model says "class 3", how often is it right? — and **recall** — of all real class 3 pixels, how many did it find? **F1** combines the two into one number. **Macro F1** is the plain average of the four class F1 scores, so **every class counts equally**.

> **Why it matters**
> A lazy model that always answers "class 2" would be right for a third of the pixels but get a macro F1 of only about 0.13. Macro F1 forces us to get **all four** age groups right, including the small ones.

3. Key words, explained simply

Word	Simple meaning
Model	A computer recipe that learns patterns from examples and then makes guesses on new data.
Training	Showing the model many examples with the right answer so it can learn the pattern.
Label	The right answer for an example (here: the age class of a pixel).
Feature	One number that describes a pixel, e.g. "average greenness over 42 years". The model only sees features.
Time series	The list of a pixel's values year after year (1984 → 2025).
Index (e.g. NDVI)	A ratio of two bands. NDVI is high for vegetation, low for buildings. Ratios ignore overall brightness.
Composite	One summary value per year made from several photos (we use the middle value, the median).
Change-point	The year a pixel's history suddenly jumps to a new level — usually when a building was put up.
Cross-validation (CV)	A fair exam: hide part of the data, train on the rest, test on the hidden part, repeat.
Spatial-block CV	The same exam, but whole 2 km × 2 km areas are hidden together, so the model cannot copy from the next-door pixel.
Overfitting	Memorising the training examples instead of learning the general pattern.
Random Forest	Hundreds of simple decision trees that each vote; the majority wins.
Gradient boosting (LightGBM, XGBoost, CatBoost)	Trees built one after another, each one fixing the mistakes of the previous ones.
Ensemble	Averaging the answers of several models — like asking several experts.
Zero-shot	Using the Madrid model in Amsterdam with no Amsterdam labels at all.
Few-shot	Adapting with only a few labelled Amsterdam pixels (5, 25, 50, 100 or 200 per class).
Support / query set	Support = the few labelled pixels we may use. Query = all other pixels, used to score us.
Domain shift	The new city looks different (Amsterdam is darker and greener than Madrid).
Class mix (prior)	The share of each class. Amsterdam has more old (class 1) and fewer new (class 4) pixels than Madrid.

Word	Simple meaning
Pseudo-label	A label the model gives itself for a pixel it is very confident about.
Per-city scaling	Describing every number relative to its own city's average — like grading each school on its own curve.
Macro F1	The average correctness score over the 4 classes (0 = useless, 1 = perfect).
GPU	A graphics card; it can train some models much faster. We used an NVIDIA RTX 4060.

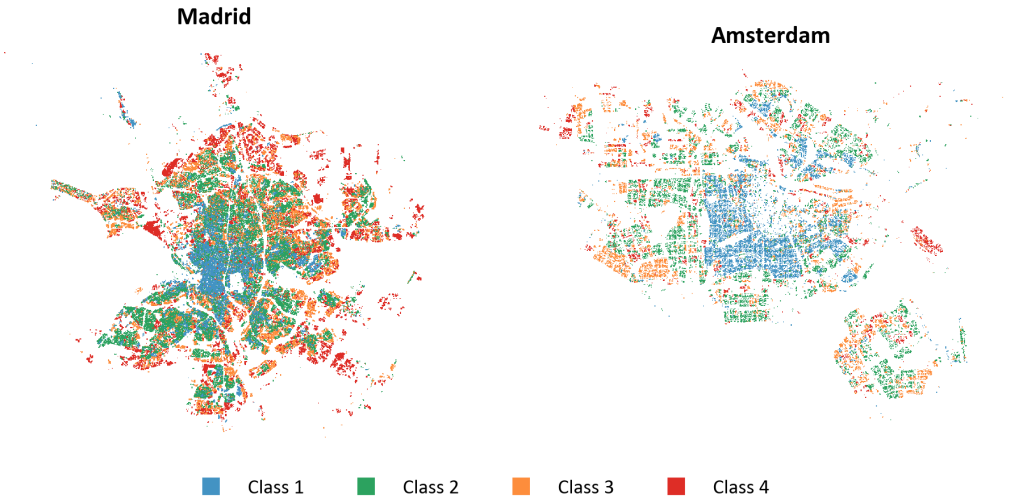
4. Stage 1 — Understanding the data

We received two files: **madrid_train.parquet** (3.17 million rows) and **amsterdam_data.parquet** (1.06 million rows). Before building anything we checked **every one of the 48 columns**. One row = **one pixel in one year**, and each row holds up to **3 photos** of that year side by side.

Column group	What it is	How we use it
Blue … SWIR2 (× 3 photos)	the six colour measurements	the input data
qa_valid, QA_PIXEL, QA_RADSAT	quality flags: cloud, shadow, snow, water, saturation	cleaning
doy (day of year), dataset	season of the photo, which satellite	season control, sensor checks
year, px_key, py_key	the year, and the pixel's map address	build the 42-year history; find neighbours
weighted_mean_year	the build year (the answer)	turned into class 1–4
coverage	share of the square covered by buildings	feature and training weight
pixel_x/y, row/col, usable_label …	copies of other columns, or always the same value	dropped

What we discovered (the starter notebooks did not)

Finding	Evidence	What we did about it
Photos are **not summer-only**	only 31 % of Amsterdam's clean photos are from June–August	prefer warm-season photos
The starter wastes good photos	it keeps only the first clean photo per year: ~60 % of clean photos discarded	use the median of all clean photos
3 whole photos are broken	every pixel far too bright, yet they pass the quality flag	remove those scenes completely
A pile of labels on 1960	2,227 Madrid pixels sit exactly on the class 1/2 boundary	tested; treated as noisy
Neighbours share their class	87 % in Amsterdam (chance 28 %)	neighbourhood features, smoothing, spatial-block CV
Different class mix	Amsterdam: +11 points class 1, −11 points class 4	class-mix correction in Amsterdam
Amsterdam looks different	darker in every band, greener	per-city scaling, ratio features



True age classes of every pixel. Old centres (blue) in the middle, newer districts (orange, red) outside: age is clustered by neighbourhood.

Say it like this

"Before modelling, we audited the data and found problems the starter notebooks missed: photos from all seasons, three broken satellite photos, and wasted data. Fixing these was our first improvement."

5. Stage 2 — Cleaning the data

Goal: one trustworthy value per pixel per year. We first turned the 3-photos-per-row layout into **one row per photo**, then removed bad photos with six rules:

Rule	Removes	Why
R1	photos not marked "clear"	cloud, shadow, snow, water, no data
R2	the 3 broken scenes	the whole photo is wrong
R3	saturated values	the sensor hit its maximum; the true value is unknown
R4	zero values	physically impossible
R5	thin high cloud (cirrus)	the organisers' flag does not check it
R6	Blue above 15 000	safety net for leftover haze

```
def clean_obs(o, bad_scenes):
    rules = [('R1 not qa_valid',   ~o.qa_valid),                 # cloud, shadow, snow, water
             ('R2 broken scene',   o.scene_id.isin(bad_scenes)),  # whole photo is wrong
             ('R3 saturated',      o.QA_RADSAT > 0),              # sensor maxed out
             ('R4 zero band',      (o[BANDS] == 0).any(axis=1)),  # impossible value
             ('R5 cirrus',         o.cirrus_conf == 3),           # thin high cloud
             ('R6 Blue>15000',     o.Blue > 15000)]               # leftover haze
    keep = np.ones(len(o), bool)
    for name, bad in rules:
        keep &= ~bad.values          # drop every photo that breaks a rule
    return o[keep]
```

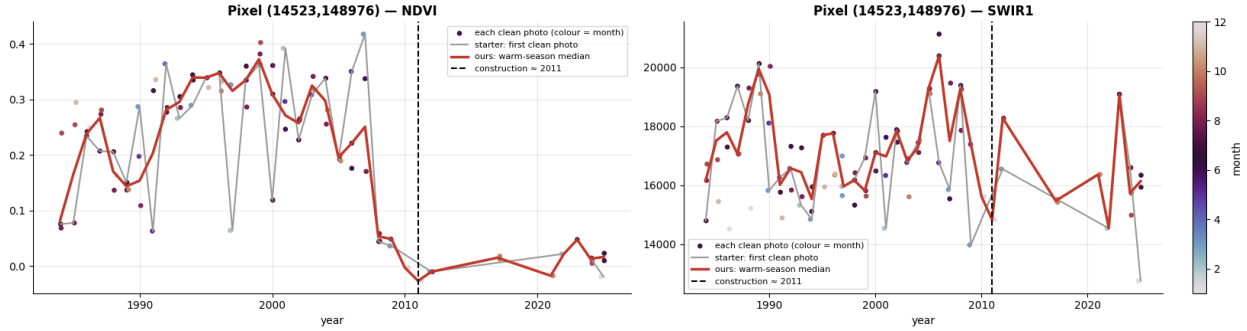
Code: the six cleaning rules (from Building_Age_Final.ipynb, Part 3).

One value per year: the warm-season median

A year can still have 1, 2 or 3 clean photos. The starter took the **first** one (often spring or winter). We take the **median** (the middle value) of all clean photos from **May to September**, and only fall back to other months when a year has none. The median ignores one strange photo; the warm-season window keeps every year comparable.

```
def yearly_composite(clean):
    warm = clean.month.between(5, 9)                                  # May-September only
    all_med  = clean.groupby(['px_key','py_key','year'])[BANDS].median()
    warm_med = clean[warm].groupby(['px_key','py_key','year'])[BANDS].median()
    use_warm = warm_med.reindex(all_med.index).notna().all(axis=1)
    return warm_med.reindex(all_med.index).where(use_warm, all_med)   # warm median, else all-year median
```

Code: building one robust value per pixel per year.



One Madrid pixel built around 2011. Dots = individual photos (colour = month); grey = starter method; red = our yearly value. Our line is smoother, so the real event — vegetation (NDVI) dropping when the site was built — stands out.

Numbers to remember

Year-to-year noise cut by **46 %** in Madrid and **40 %** in Amsterdam.
Photos kept after cleaning: **92.5 %** (Madrid), **71.6 %** (Amsterdam — a cloudy city).
With the starter's own features, cleaning alone lifted Madrid from **0.587 to 0.599**.

6. Stage 3 — Feature engineering

In plain words

A model cannot look at a 42-year history directly. We describe each pixel with a list of numbers — its **features** — like summarising a person's 42-year health record with "average weight", "biggest change", "year of the biggest change". We made **321 features** per pixel, all computed **without** knowing the answer.

The guiding idea: describe **how a pixel changes**, not **how bright it is**. Brightness differs between cities (sunny Madrid, cloudy Amsterdam), but "a building appeared in 2006" looks the same everywhere. The challenge rules explicitly encourage teams to "engineer additional features, apply spatial context, or use any other strategy".

Group	Features	What it captures	Why it helps
S Starter-like	58	averages, spreads, before vs after 2004, yearly change	baseline description
A New indices	20	NBR, NDTI, tasselled-cap brightness/greenness/wetness	more material and vegetation detail

Group	Features	What it captures	Why it helps
B Robust trends	88	median, middle range, slopes, how jumpy	one bad year cannot distort them
C Time windows	35	medians of 6-year periods	tells **when** the pixel changed
D Change-point	36	year, size and strength of the main jump	construction leaves a jump (classes 3–4)
E Relative	23	late minus early, ratios, colour shape	cancels city brightness
F Quality	5	building coverage, missing years	how much to trust the pixel
G Spatial	34	averages of the neighbouring pixels	districts are built in the same era
H Two-break	22	two jumps (demolish → rebuild); steepest green-to-built change	added after research; helps every n

6.1 Spectral indices — ratios that ignore brightness

```
cbd['NDVI'] = (N - R)  / (N + R  + e)       # vegetation: high = green, low = buildings
cbd['NDBI'] = (S1 - N) / (S1 + N + e)       # built-up surfaces
cbd['BSI']  = ((S1 + R) - (N + B)) / ((S1 + R) + (N + B) + e)   # bare soil = construction sites
cbd['NBR']  = (N - S2) / (N + S2 + e)       # vegetation vs bare ground
```

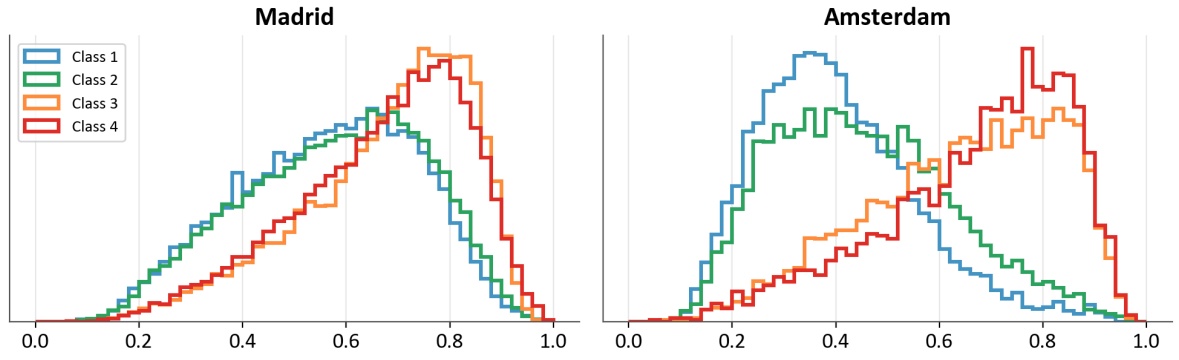
Code: a few of the 10 indices. A ratio stays the same if the whole picture gets brighter or darker.

6.2 Change-point features — finding the year a building appeared

For each pixel we try every possible split year and ask: "if the history were flat before this year and flat after it, how much of the variation would that explain?" The best split gives the **year** of the jump, its **size**, and its **strength** (0 = no jump, 1 = perfect step).

```
def step_break(Y, years):                    # Y = pixels x 42 years
    T = Y.shape[1]; k = np.arange(1, T)      # every possible split point
    cs = np.cumsum(Y, axis=1)[:, :-1]
    before = cs / k                          # average level before the split
    after  = (Y.sum(1, keepdims=True) - cs) / (T - k)   # average level after it
    gain = k * (T - k) / T * (after - before) ** 2      # how much a step explains
    best = gain.argmax(1)                    # the best split year for each pixel
    strength = gain.max(1) / (Y.var(1) * T)  # 0 = no step, 1 = perfect step
    return years[best + 1], (after - before)[np.arange(len(Y)), best], strength
```

Code: the one-break change-point, computed for all pixels at once (group D). Group H does the same with two breaks.



Classes 3 and 4 (orange, red — built after 1984) have strong jumps; classes 1 and 2 (blue, green) do not. The gap is even clearer in Amsterdam, so this signal travels between cities.

6.3 Spatial context — looking at the neighbours

Buildings in one district are usually built in the same era. For key features we also compute the **average of the surrounding pixels** in squares of 90 m, 210 m and 450 m, leaving the pixel itself out. We only use the neighbours' **features**, never their answers.

```
for f in ['NDVI_med', 'NDBI_med', 'NDVI_brk_year', 'coverage', ...]:
    for size in (3, 7, 15):                    # 3x3, 7x7, 15x15 pixels = 90, 210, 450 m
        out[f'nb{size}_{f}'] = neigh(pixels, base[f], size)   # neighbourhood average
```

Code: neighbourhood features (group G).

6.4 Per-city scaling — grading each city on its own curve

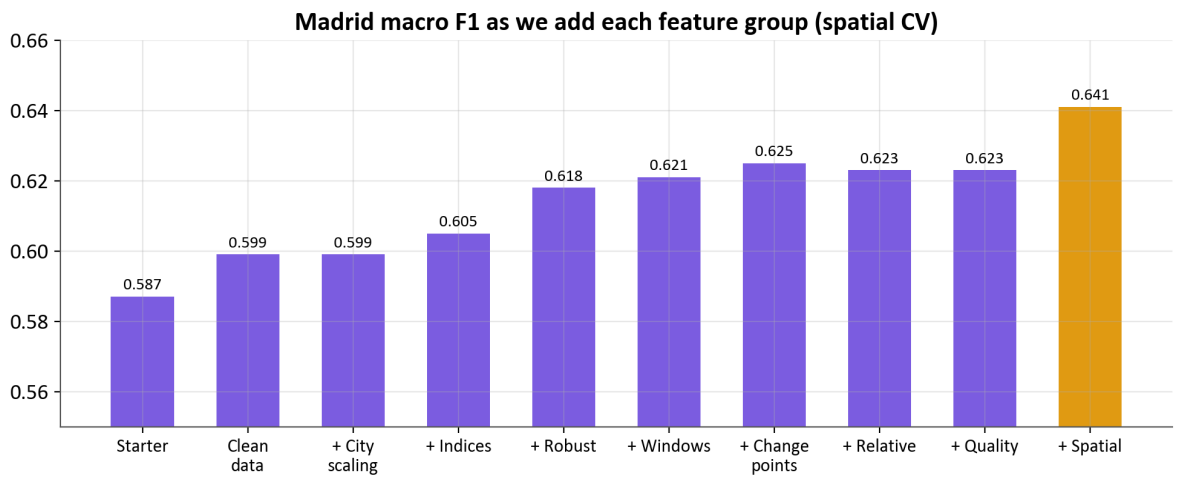
Every feature is converted into "how far above or below this city's average". Amsterdam pixels are compared with Amsterdam, Madrid pixels with Madrid. This uses no labels and was the single biggest improvement for transfer.

```
def city_scale(X):                          # applied separately to Madrid and to Amsterdam
    mu = np.nanmean(X, 0); sd = np.nanstd(X, 0)
    return (X - mu) / sd                    # 0 = typical for this city, +1 = one spread above
```

Code: per-city standardisation.

6.5 How we proved each group helps (ablation)

We added one group at a time, keeping everything else fixed, and measured the honest Madrid score each time:



Every group raised Madrid accuracy: 0.587 → 0.641. Spatial context gave the largest single step.

Why it matters

Per-city scaling raised the Amsterdam no-label score from **0.413 to 0.520** without any labels.
But adding features did **not** raise the Amsterdam 25-label score (it stayed near 0.60). That told us the next improvement had to come from the **transfer method** (chapter 8), not more features.
Spatial features help in Madrid but **hurt** when moving to Amsterdam, because district patterns differ between cities. So we use two feature sets: a **rich set** (best 120 features) for Madrid and a **transfer set** (140 city-independent features) for Amsterdam.

7. Stage 4 — Training the Madrid model

In plain words
"Training" means showing the model the 321 features of tens of thousands of Madrid pixels **together with their true age class**, so it learns which feature patterns go with which age.

7.1 The models we compared

Model	Simple picture	Special preparation
Random Forest (baseline)	300 decision trees each vote; majority wins	missing values filled with the city average
Logistic Regression	one weighted sum per class — the simplest model	missing values filled; per-city scaling required
MLP (neural network)	layers of small calculators (128 → 64), runs on the GPU	missing values filled; scaling required
LightGBM	trees built in sequence, each fixing earlier mistakes	handles missing values itself
XGBoost	the same idea, different implementation (GPU)	handles missing values itself
CatBoost	the same idea, very careful against overfitting (GPU)	handles missing values itself
Ensemble	average of the top 3 models' answers	—

Fair weighting: class 2 has twice as many pixels as class 1. Without help, a model would favour class 2. We give each training pixel a weight = **class balance** × **building coverage**, so small classes count as much as big ones and mostly-building pixels (cleaner signal) count a bit more.

```
class Model:                                  # one interface for all six algorithms
    def __init__(self, name):
        self.impute = name in ('Random Forest', 'Logistic Regression', 'MLP neural net')
        if name == 'Random Forest':
            self.m = RandomForestClassifier(n_estimators=300, min_samples_leaf=2, n_jobs=N_JOBS)
        elif name == 'LightGBM':
            self.m = lgb.LGBMClassifier(n_estimators=400, learning_rate=0.05, num_leaves=63)
        elif name == 'CatBoost':
            self.m = cb.CatBoostClassifier(iterations=400, depth=6, task_type=CAT_TASK)  # GPU if found
        ...
    def fit(self, X, y, w):                   # w = class-balance x coverage weights
        self.m.fit(np.nan_to_num(X) if self.impute else X, y, sample_weight=w)
        return self
```

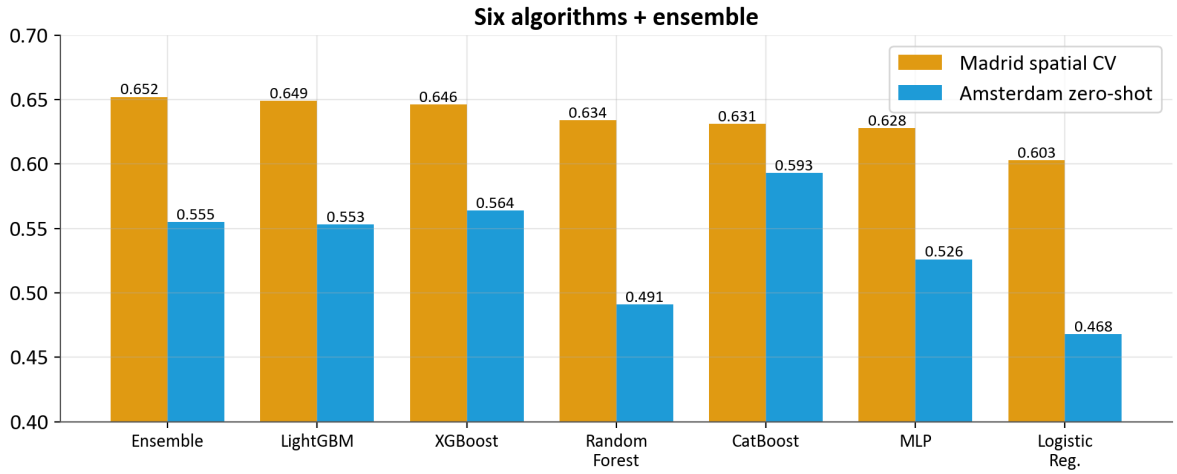
Code: every algorithm gets its own settings and preprocessing, behind one common interface.

7.2 Testing honestly: spatial-block cross-validation

In plain words
Imagine an exam where the student sits next to someone who has the answers. Neighbouring pixels are nearly identical, so if a pixel is in the test and its neighbour is in training, the model just "copies". We cut Madrid into **2 km × 2 km blocks** and hide **whole blocks** at a time. The test pixels have no close neighbours in training — a fair exam.

```
blocks = (px_key // 67).astype(str) + '_' + (py_key // 67).astype(str)   # 67 px x 30 m = 2 km
CV = StratifiedGroupKFold(5, shuffle=True, random_state=42).split(X, y, groups=blocks)
for train_idx, test_idx in CV:
    m = Model(name).fit(X[train_idx], y[train_idx], W[train_idx])
    scores.append(macro_f1(y[test_idx], m.predict(X[test_idx])))
print(np.mean(scores), '+/-', np.std(scores))
```

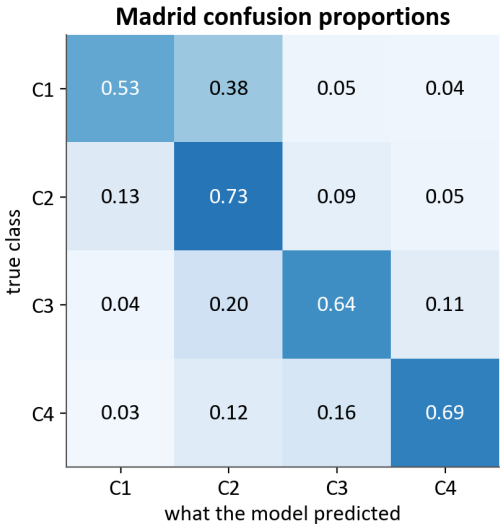
Code: 5-fold spatial-block cross-validation.



Madrid score (amber) and Amsterdam no-label score (blue) for each algorithm.

Result	Value	What it tells us
Random Forest baseline	0.634	a solid start
Best single model (LightGBM)	0.649	boosting beats a forest
Ensemble of top 3	**0.652 ± 0.026**	our Madrid model
CatBoost in Amsterdam (no labels)	**0.593**	travels best → base of the Amsterdam model
Same ensemble, ordinary random CV	0.736 ± 0.003	looks better but is inflated by neighbour copying

7.3 Where the Madrid model makes mistakes



Read each row: of all true class-1 pixels, 53 % were called class 1 and 38 % class 2.

Most mistakes happen between **neighbouring** classes (1↔2, 3↔4). Class 1 vs 2 is hardest: both were built before 1984, so neither shows a construction jump in the satellite record.

Say it like this

"We started with Random Forest, compared five more algorithms, and an ensemble of the three best reached 0.652 under strict spatial validation. Ordinary validation would have reported 0.736, but that number is inflated by neighbouring pixels, so we report the honest one."

8. Stage 5 — Moving the model to Amsterdam

8.1 Why this is hard

- **Amsterdam looks different:** darker in every band and greener (water, brick, trees). A Madrid model reads "darker" as "older" and gets confused.
- **Different class mix:** Amsterdam has many more old pixels and far fewer new ones.
- **Very few labels:** as few as 5 known pixels per class.

8.2 With no labels at all (zero-shot)

Step	Amsterdam score
Starter-style scaling	0.519
Per-city scaling (CatBoost)	0.593
+ class-mix correction (EM) without labels	0.459 (worse — see 8.4)
Per-city scaling + neighbourhood smoothing (used)	**0.624**

8.3 With a few labels: our four transfer ingredients

For each test we randomly pick n labelled Amsterdam pixels per class (the **support set**) and score on all the others (the **query set**). We repeat with 10 different random picks.

Ingredient	Simple picture	Uses Madrid?	Uses unlabelled Amsterdam?
Prototype (starter method)	average the support pixels of each class; nearest average wins	no	no
Recalibration	keep the Madrid model, but learn from the support set how to re-read its answers	yes	no
Fine-tune	retrain on all of Madrid plus the support pixels, which count very heavily	yes	no
Self-training	take the 500 pixels per class the model is most sure about as extra "pseudo-labels" and retrain	yes	yes
FINAL	average of the three, then class-mix correction (only if n ≥ 25), then neighbourhood smoothing	yes	yes

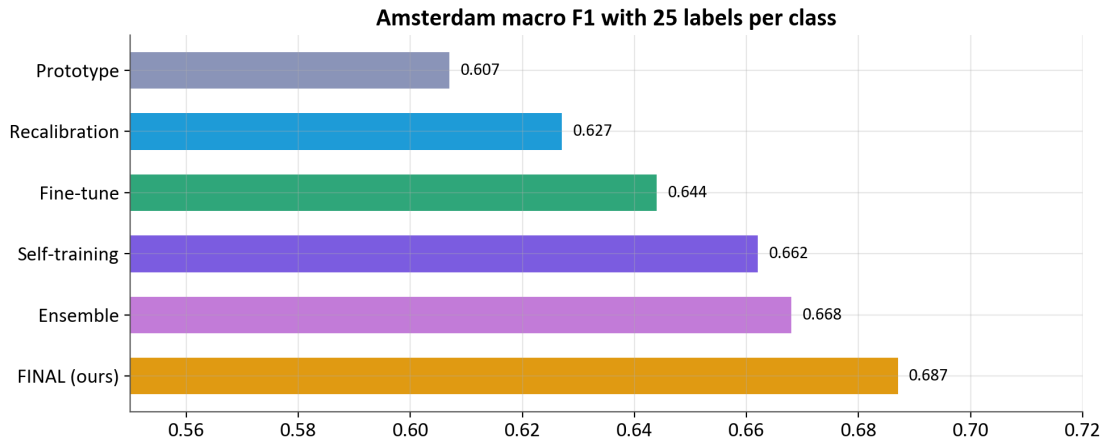
```
def adapt_to_city(s, y_s):                    # s = support pixels, y_s = their classes
    recal = LogisticRegression().fit(logPZ[s], y_s).predict_proba(logPZ)      # 1 recalibrate
    ft = lgb.LGBMClassifier(**FT_PARAMS).fit(np.vstack([XM, XA[s]]), np.r_[ym, y_s],
            sample_weight=np.r_[W_MAD, heavy_weight]).predict_proba(XA)       # 2 fine-tune
    pseudo = most_confident(recal + ft, per_class=500)                       # 3 self-train
    st = lgb.LGBMClassifier(**ST_PARAMS).fit(XA[s + pseudo], labels).predict_proba(XA)
    ens = (recal + ft + st) / 3                                               # ensemble
    if len(s) // 4 >= 25:
        ens = em_prior(ens)[0]               # safe class-mix correction, only with enough labels
    return smooth_proba(ens, pixels)         # blend with neighbours (tuned on Madrid)
```

Code: the Stage-2 adaptation, simplified from adapt_to_city() in the final notebook.

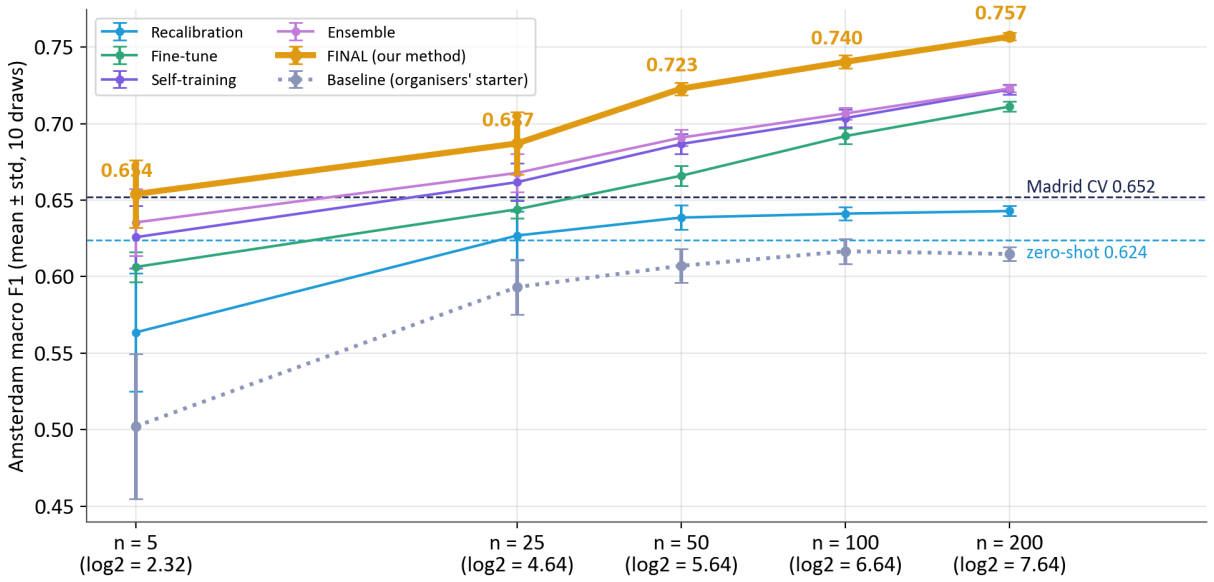
8.4 The class-mix correction and why it is "safe"

The model was trained with balanced classes, so it assumes 25 % of each class. Amsterdam is different. The **EM** method estimates Amsterdam's real mix from the model's own guesses and re-weights them. It only works when the guesses are reliable: with no labels it estimated **0 % class 1** (truth 29 %) and made things worse; with 5 labels it was still unstable. So we apply it **only when there are at least 25 labels per class**. This one rule lifted n = 5 from 0.597 to 0.654.

8.5 Results



At 25 labels per class every ingredient adds on top of the simple prototype method; the full method beats the organisers' starter baseline (0.593) by +0.094.



The required plot: Amsterdam score (mean ± std over 10 draws) against log2 of the labels per class. Our method (amber) is best at every size.

Say it like this

"The starter method ignores the Madrid model. We reuse it three ways — recalibrating it, fine-tuning it with Amsterdam examples, and letting it teach itself from thousands of unlabelled Amsterdam pixels. With only 5 labels per class we already match Madrid's own score."

9. Stage 6 — Testing ideas from research papers

After our first full run we looked for weak spots (the 5-label score, the no-label class-mix estimate) and searched the research literature. We tested six ideas on **exactly the same 10 random draws**, on two computers, and kept only what helped.

Idea	Source	Result	Decision
Safe class-mix correction (only when n ≥ 25)	Alexandari et al., ICML 2020	n = 5: 0.597 → 0.654, spread halved	**Kept**
Two-break change features (group H)	BATSCCD, Int. J. Digital Earth 2024	+0.004 to +0.011 at every n	**Kept**
TabPFN v2 (a pre-trained "foundation" model for small tables)	Hollmann et al., Nature 2025	+0.003 — inside the error bars	Not used
Several rounds of self-training	TimeMatch, ISPRS 2022	same as one round	Rejected
Calibrating before the class-mix correction	Alexandari et al., ICML 2020	zero-shot still broken	Rejected
CORAL (forcing Amsterdam's statistics to look like Madrid's)	Sun et al., AAAI 2016	−0.05 at n = 25	Rejected

Why it matters
Showing what did **not** work, with numbers, proves we tested carefully instead of guessing. TabPFN was left out on purpose: it needs a GPU, older library versions and an internet download — a risk for judges re-running our code, for a gain smaller than the error bars.

The final check: pre-registered experiments (notebook 03)

The ideas above were checked on the same 10 Amsterdam draws we report. That carries a small risk called **selection bias**: if you pick what looks best on your test, the reported score can be a little optimistic. On the last night we removed that risk with a stricter protocol, written down **before** running anything:

- Amsterdam was cut into **2 km blocks**; half the blocks became **DEV** (for decisions), the other half **TEST** (never looked at until the end).
- Every idea was judged on DEV only, with new random draws (seeds 100–109). Madrid ideas were judged on Madrid spatial-block CV only.
- **Adoption rule:** the gain must be larger than **2 standard errors**, and pixels at least 1 km from any label must not get worse.
- The final model was then scored **once** on TEST: labels taken from DEV blocks, scores measured only in TEST blocks.
- Everything was run on two computers (this PC on CPU, the Legion on GPU).

Idea (what it targets)	Gain on DEV	2 × SE	Decision
E1 Label-noise weights: pixels within ±2 years of a class boundary count half (1960 pile)	+0.001	0.002	Rejected — no gain (Madrid: 0.651 vs 0.652)
E2 BBSE class-mix estimate instead of EM (Lipton et al., ICML 2018)	−0.002 to −0.003	0.002	Rejected — worse
E2c EM class-mix correction at every n	−0.003	0.003	Rejected — confirms our n ≥ 25 rule
E3 Agreement filter for pseudo-labels	+0.0004 (Legion) / −0.001 (PC)	0.0004	Rejected — no gain on TEST (−0.0007)
E4 Weight of Amsterdam labels in fine-tuning: 10 % or 50 % (we use 25 %)	−0.002 / 0.000	0.001	Rejected — 25 % stays
E5 Turning spatial smoothing off	**−0.020**	0.001	Smoothing clearly earns its place

Result on the untouched TEST half (labels from the other half of the city, 10 draws):

Labels per class	5	25	50	100	200	Average
Unseen half of Amsterdam	0.652 ± 0.031	0.705 ± 0.008	0.717 ± 0.011	0.729 ± 0.008	0.748 ± 0.004	**0.710**
Pixels ≥ 1 km from any label	0.648	0.703	0.717	0.731	0.760	0.712
Official protocol (what we report)	0.654 ± 0.021	0.687 ± 0.019	0.723 ± 0.004	0.740 ± 0.004	0.757 ± 0.002	**0.712**

What this proves

Our reported scores are not inflated: on districts the labels never touched, the average is the same (0.710 vs 0.712).

The model does not copy nearby labels: pixels more than 1 km from any label score just as well.

The protocol works: idea E3 passed on DEV by the narrowest margin on one computer, failed on the other, and gave nothing on TEST — a false positive the held-out half caught.

Because nothing passed the rule, **the submitted model is unchanged**.

10. Final results, and why we keep them

Setting	Labelled pixels	Our score (mean ± std)	Starter baseline
Madrid — spatial-block 5-fold CV	all	**0.652 ± 0.026**	0.574 ± 0.030
Madrid — random 5-fold CV	all	0.736 ± 0.003	0.618 ± 0.005
Amsterdam — zero-shot	0	0.624	0.334
Amsterdam — n = 5	20	**0.654 ± 0.021**	0.502 ± 0.047
Amsterdam — n = 25	100	**0.687 ± 0.019**	0.593 ± 0.018
Amsterdam — n = 50	200	**0.723 ± 0.004**	0.607 ± 0.011
Amsterdam — n = 100	400	**0.740 ± 0.004**	0.617 ± 0.008
Amsterdam — n = 200	800	**0.757 ± 0.002**	0.615 ± 0.004
Average of the 5 Amsterdam models	—	**0.712**	0.587

Starter baseline = the organisers' pipeline (first photo per year, their 60 features, Random Forest, prototype transfer), re-run in our submission notebook on the same folds and draws. It reproduces their own reported figures (random CV ≈ 0.62, zero-shot 0.343).

Why we keep these choices

Choice	Why
Report the **spatial** Madrid score (0.652), not the random one (0.736)	The hidden test pixels are new areas; the spatial score predicts that honestly. The random score is inflated by neighbour copying.
Ensemble (LightGBM + XGBoost + Random Forest) for Madrid	Best Madrid score, and averaging makes predictions more stable.
CatBoost as the Amsterdam base model	Best score in Amsterdam with no labels: it travels best between cities.
FINAL = ensemble + safe class-mix correction + smoothing	Best at every sample size, smallest error bars, nothing tuned on Amsterdam answers.
Label propagation variant not used as the main result	Slightly higher (+0.004) but depends on test pixels sitting next to labelled ones; less safe for hidden test sets.
TabPFN not used	Gain inside the error bars; reproducibility risk.
Per-city scaling everywhere	Biggest gain that needs no labels; works for any new city.

Why we trust the numbers

- **10 random draws** for each sample size, with fixed seeds — anyone can reproduce them.
- **Two computers, same answer:** the CPU run and the RTX 4060 GPU run agree within ±0.003.
- **Unseen-half test:** with labels from one half of Amsterdam and scores measured only on the other half, the average is **0.710** (reported 0.712), and pixels at least 1 km from any label score just as well (0.65–0.76). The gain is not copying neighbours (chapter 9).
- **Same draws for every method**, so every comparison is like-for-like.

Honest limitations

- Class 1 vs class 2 remains the hardest pair: both pre-date the satellite and show no construction jump.
- The labels are averages over several buildings, so some pixels are genuinely mixed; no model can be perfect.
- We used Amsterdam's unlabelled pixels (self-training). This is realistic — satellite images exist for every city — but we have asked the organisers to confirm it is allowed.

11. Questions the judges may ask — with answers

Q: Your Madrid model scores 0.736 with normal cross-validation. Why do you report 0.652?

A: Normal (random) cross-validation lets the model copy from near-identical neighbouring pixels, which inflates the score. The hidden test pixels are new areas, so we report the stricter spatial-block score (0.652). For comparison, the organisers' starter baseline scores 0.618 with random validation and 0.574 with spatial validation.

Q: Why can Amsterdam (0.740 at n = 100, 0.757 at n = 200) score higher than Madrid?

A: First, it is **not leakage**: we checked by taking labels from one half of Amsterdam and scoring only the other half — still 0.710 on average — and pixels more than 1 km from any label score just as well. The real reasons: (1) **Amsterdam is an easier city for this task:** its class 1/2 boundary (1945) falls in a wartime construction gap (only 124 pixels within ±2 years), while Madrid's (1960) sits on a pile of 2,227 labels (5,523 within ±2 years), so far more Madrid pixels are borderline. (2) **More help:** the Amsterdam model starts from the trained Madrid model, adds Amsterdam's own labels and learns from thousands of unlabelled Amsterdam pixels. The starter baseline shows the same pattern (Madrid random CV 0.618, Amsterdam n = 200 0.615).

Q: How do you know you did not overfit to your test numbers?

A: On the last night we wrote the decision rules down first, split Amsterdam into a decision half and an untouched half, tested eight ideas on the decision half only, and scored the final model once on the untouched half: 0.710 against the reported 0.712. None of the eight ideas passed, so the submitted model is unchanged.

Q: Isn't using unlabelled Amsterdam pixels cheating?

A: No answers are used — only satellite measurements, which exist for every city. Every score is measured on pixels whose labels the model never saw.

Q: How do you know your features are not just memorising Madrid?

A: We test in Amsterdam, a different city. Features that only memorised Madrid (like spatial ones) lowered the Amsterdam score, so we kept them out of the transfer set.

Q: What is the single most important idea?

A: Describing **change over time** instead of brightness, plus scaling each city on its own.

Q: Why did you not use deep learning everywhere?

A: We tested a neural network on the GPU (0.628). Tree models were better on this tabular data and much faster.

Q: Why is n = 5 the least stable?

A: With 20 labels, results depend on which pixels happen to be picked. Our safe rule halved the spread (±0.061 → ±0.022).

Q: What would you do with more time?

A: Pilot the method in an African city such as Harare with about 100 labelled pixels, add Sentinel-2 (10 m) images for recent years, and revisit TabPFN-style models.

Q: How long does it take to run?

A: 18 minutes on an RTX 4060 laptop GPU, about 60 minutes on a normal CPU, from raw data to final results.

Q: Which data problems did you find?

A: Photos from all seasons, three broken satellite photos that pass the quality flag, 60 % of clean photos wasted by the starter, and a pile of labels exactly on 1960.

Q: How did you handle class imbalance?

A: Balanced training weights, macro-F1 for every decision, and a class-mix correction in Amsterdam when there are enough labels.

Q: Why CatBoost for Amsterdam but the ensemble for Madrid?

A: Each was best at its own job: the ensemble in Madrid, CatBoost when moving to a new city with no labels.

12. Who presents what, slide by slide

We present from **Group2_SCREEN_Presentation.pptx** (12 slides, 8 minutes). Each presenter has a cue card in **Group2_TEAM_Cue_Cards.pptx**.

Time	Presenter	Slides	Key sentence to land
0:00–1:30	Precious (GIS)	1–3 Title, city from orbit, data findings	"We date every 30 m square of a city from 42 years of satellite photos, then move to a new city with very few labels."
1:30–2:45	Prof Tanaka (physics)	4 Physics of building age	"We describe how each square changes, not how bright it is — that is what travels between cities."
2:45–4:15	Natali (computer science)	5 How we built it	"Five steps, honest 2 km spatial testing, and CPU and GPU agree within 0.003."
4:15–7:45	Winston (data science)	6–12 Models, transfer, results, plot, honest check, lessons, thanks	"With 25 labels per class we reach 0.687, +0.094 over the starter; on districts the labels never touched we still score 0.71."

Presenting tips

Speak to the plot on slide 9: point at the amber line (ours), then at the grey dotted line (starter), then at the dashed line (Madrid score).
On slide 10, point at the map: amber = where labels came from, blue = where we scored.
Thank the partners on slide 12 by name: Zimbabwean Astronomical Society, ZINGSA, AfAS, OAD, IDIA and DARA.

13. Our files and how to run them

File	What it is
Building_Age_Final.ipynb	the final notebook: every stage, executed, with explanations for the judges
submission/stage1_madrid_rich_Ensemble_top-3.joblib	the trained Madrid model (deliverable 1)
submission/stage1_transfer_base.joblib	the trained CatBoost transfer base model
submission/required_results.csv, f1_vs_log2_n.png	the required results table and plot
ABSTRACT.md, WRITTEN_JUSTIFICATION.md	the 150-word abstract and 300-word justification
Group2_SCREEN_Presentation.pptx / Group2_TEAM_Cue_Cards.pptx	the 12-slide screen deck and the team cue cards
Group2_Building_Age_Presentation.pptx	the longer 19-slide reference deck
supporting_00 / 01 / 02 notebooks	data audit, feature ablation, research experiments
03-Honest_Improvements.ipynb + exp03_outputs/	the pre-registered experiments and the unseen-half test (chapter 9)
Group2_QA_Broken_Scenes_and_Spatial_Blocks.pdf	two-page explainer: broken scenes and 2 km spatial blocks

To re-run everything

- Put **madrid_train.parquet** and **amsterdam_data.parquet** next to the notebook.
- Install the packages: **pip install -r requirements.txt** (plus PyTorch with CUDA for the GPU; without a GPU everything falls back to the CPU automatically).
- Open the notebook and choose **Run All** — or double-click **RUN_ME.bat** on the GPU laptop. About 18 minutes on the RTX 4060, about 60 minutes on a CPU.
- The adaptation is parameterised by n and uses fixed random seeds (42 for models, 0–9 for the 10 draws), so the numbers are reproducible.

14. References

- Alexandari, Kundaje & Shrikumar (2020). Maximum likelihood with bias-corrected calibration is hard-to-beat at label shift adaptation. ICML.
- Hollmann et al. (2025). Accurate predictions on small data with a tabular foundation model (TabPFN v2). Nature.
- Nyborg et al. (2022). TimeMatch: unsupervised cross-region adaptation by temporal shift estimation. ISPRS Journal of Photogrammetry and Remote Sensing.
- Roberts et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. Ecography.
- Roy et al. (2016). Characterization of Landsat-7 to Landsat-8 reflective wavelength and NDVI continuity. Remote Sensing of Environment.
- Saerens, Latinne & Decaestecker (2002). Adjusting the outputs of a classifier to new a priori probabilities. Neural Computation.
- Sun, Feng & Saenko (2016). Return of frustratingly easy domain adaptation (CORAL). AAAI.
- BATSCCD (2024). A change detection method for mapping building age using Landsat time series. International Journal of Digital Earth.
- Snell, Swersky & Zemel (2017). Prototypical networks for few-shot learning. NeurIPS.